02. Captura de Datos RealSense (`escanear.py`)

Este documento profundiza en la lógica del script de adquisición de datos crudos utilizando la cámara Intel RealSense D455 y la librería Open3D.

1. Inicialización del Sensor

El script depende de la nueva API Tensors de Open3D para trabajar fluidamente con cámaras RGB-D.

```
rscam = o3d.t.io.RealSenseSensor()
rscam.init_sensor(o3d.t.io.RealSenseSensorConfig())
rscam.start_capture()
```

Esto establece la comunicación USB con la D455 y arranca los emisores infrarrojos para la reconstrucción estéreo activa.

2. El Bucle de Captura en Tiempo Real

El código levanta una ventana de visualización gráfica (`o3d.visualization.Visualizer()`) y entra en un bucle no bloqueante (`wait=False`). Dentro del bucle se captura un `rgbd_frame`:

```
rgbd_frame = rscam.capture_frame(wait=False, align_depth_to_color=True)
```

Importancia de `align_depth_to_color=True`: El sensor de profundidad y el de color no están físicamente en el mismo lugar en la cámara (hay un desfase o baseline). Este parámetro asegura que cada píxel de profundidad corresponda exactamente al mismo píxel de la imagen a color, re-proyectando las imágenes por hardware internamente antes de entregarlas a Python.

3. Conversión de RGB-D a Nube de Puntos (3D)

Con los fotogramas alineados y los parámetros intrínsecos de la cámara (`rscam.get_metadata().intrinsics`), se reconstruye el entorno geométrico usando matemática tensorial:

```
current_pcd_t = o3d.t.geometry.PointCloud.create_from_rgbd_image(
    rgbd_frame, intrinsic_tensor
```

```
)
```

Esto genera las coordenadas (X, Y, Z) para cada píxel y lo colorea basándose en la imagen RGB. Estos puntos se envían a la memoria de la tarjeta gráfica (o CPU) para refrescar el visualizador en vivo (`vis.update_geometry()`).

4. Guardado de los Datos Finales

Cuando el usuario cierra la ventana, se captura la excepción o se continúa en el bloque `finally`, invocando la detención del sensor de manera segura (`rscam.stop_capture()`) y volcando el resultado actual al disco duro usando formato `.ply` (Polygon File Format). En la versión integrada, esto también incluye volcar metadatos JSON y texturas JPG para el procesamiento offline.